

AI STUDYBUDDY – AI ASSISTANT FOR SMARTER LEARNING AND STUDY SUPPORT

Submitted by

TEAM LEADERS : HARINI S

TEAM MEMBERS : DEEPIKA M , GOPIKA R , SATHANA M , SANGEETHA V

TABLE OF CONTENTS

1. Introduction	3
2. Problem Statement	3
3. Proposed Solution	3
4. Usage and Roles	3
5. Requirements	3
6. Technology Stack	4
7. Entity–Relationship (ER) Diagram	5
8. MVC Architecture	6
9. User Flow	7
10. Project Setup and Configuration	8
11. Backend Development	8
12. Database Configuration and API Endpoints	9
13. Project Execution and API Testing	10
14. Future Enhancements	14
15. Conclusion	14

1. Introduction

AI StudyBuddy is an AI-powered backend application that helps students study smarter. A student can upload study material and the system automatically generates a summary, flashcards, a quiz and a personalised study plan using the Google Gemini AI API.

The backend is built with Node.js and Express.js as a RESTful API and stores data in MongoDB using Mongoose. Users are secured with JWT (JSON Web Token) authentication and Role-Based Access Control (Student and Admin). Passwords are encrypted with bcryptjs.

2. Problem Statement

Students face the following difficulties while preparing for exams:

- Lengthy study materials are hard to understand in limited time.
- Making notes, flashcards and questions manually is slow and repetitive.
- No personalised study plan based on available time and exam dates.
- No single secure place to store learning resources.

3. Proposed Solution

AI StudyBuddy provides one centralised platform where a logged-in student can upload study material and let Gemini AI create learning resources automatically. The backend:

- Authenticates users with JWT and protects private routes.
- Stores users and study materials in MongoDB.
- Sends the material to the Gemini model through a separate AI service (utils/gemini.js).
- Generates summaries, flashcards, multiple-choice quizzes and study plans, and saves them for later revision.

4. Usage and Roles

- **Student:** registers, logs in, uploads study material, generates summary, flashcards, quiz and study plan, and views previously generated content.
- **Administrator:** registers and logs in as admin, manages registered users and monitors the system using admin-only APIs protected by role-based access control.

5. Requirements

Software Requirements

- **Operating System:** Windows 10/11, macOS or Linux
- **Node.js (v16+) and npm:** runtime and package manager
- **MongoDB:** database for users and study materials
- **Google Gemini API key:** AI content generation
- **Postman:** API testing

- **Visual Studio Code:** code editor

Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen) / AMD Ryzen 5 or equivalent
- **RAM:** 8 GB minimum (16 GB recommended)
- **Storage:** 1 GB free disk space

6. Technology Stack

The following technologies are used to build AI StudyBuddy:

Layer	Technology
Frontend	React.js
Backend	Node.js, Express.js
Database	MongoDB with Mongoose ODM
Authentication	JWT, bcryptjs, Role-Based Access Control
File Upload	Multer
AI Model	Google Gemini 2.5 Flash (@google/genai)
Testing Tool	Postman

7. Entity–Relationship (ER) Diagram

The project uses two MongoDB collections: User and Material. The AI-generated results (summary, flashcards, quiz and study plan) are stored inside the Material document as embedded fields.

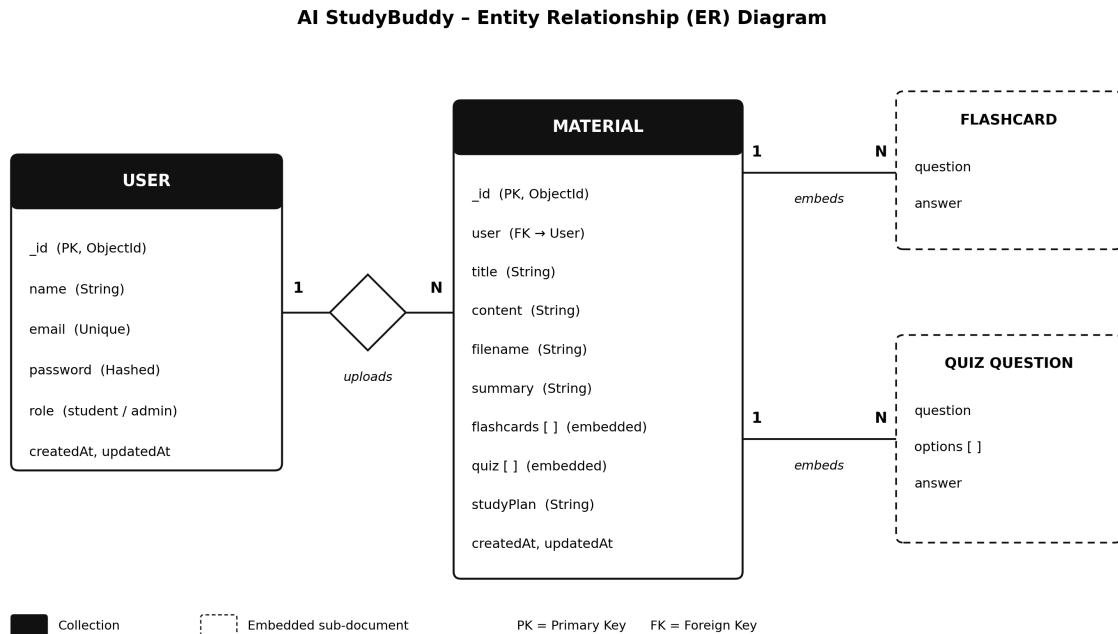


Figure 1: ER Diagram of AI StudyBuddy

Entities

- **User:** `_id` (PK), `name`, `email` (unique), `password` (hashed), `role` (student / admin), `createdAt`, `updatedAt`.
- **Material:** `_id` (PK), `user` (FK → User), `title`, `content`, `filename`, `summary`, `flashcards [ ]`, `quiz [ ]`, `studyPlan`, `createdAt`, `updatedAt`.
- **Flashcard (embedded):** `question`, `answer`.
- **Quiz question (embedded):** `question`, `options [ ]`, `answer`.

Relationships

- **User → Material (One-to-Many):** one user can upload many study materials.
- **Material → Flashcards / Quiz (One-to-Many):** one material contains many flashcards and quiz questions.
- **Material → Summary / Study Plan (One-to-One):** each material stores one summary and one study plan.

8. MVC Architecture

The project follows the Model–View–Controller pattern, which separates data, logic and responses. Every request passes through the routes and JWT middleware to the controller, which uses the models and the Gemini AI service and returns a JSON response.

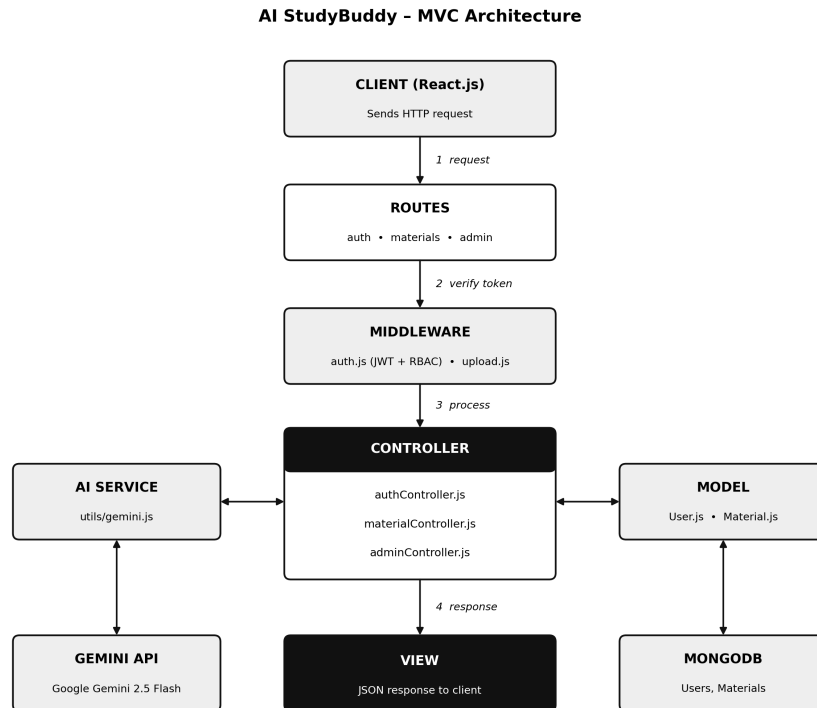


Figure 2: MVC Architecture of AI StudyBuddy

- **Model:** models/User.js and models/Material.js define the Mongoose schemas and handle data validation and database access.
- **View:** the backend has no user interface; it returns JSON responses to the React client.
- **Controller:** authController.js, materialController.js and adminController.js receive requests, apply the logic, call the models and Gemini, and send the response.
- **Benefit:** the code is organised, easy to test and easy to extend with new features.

9. User Flow

The user registers or logs in and receives a JWT token. After login the user uploads study material, selects an AI feature, and the result generated by Gemini is saved in MongoDB and shown to the user.

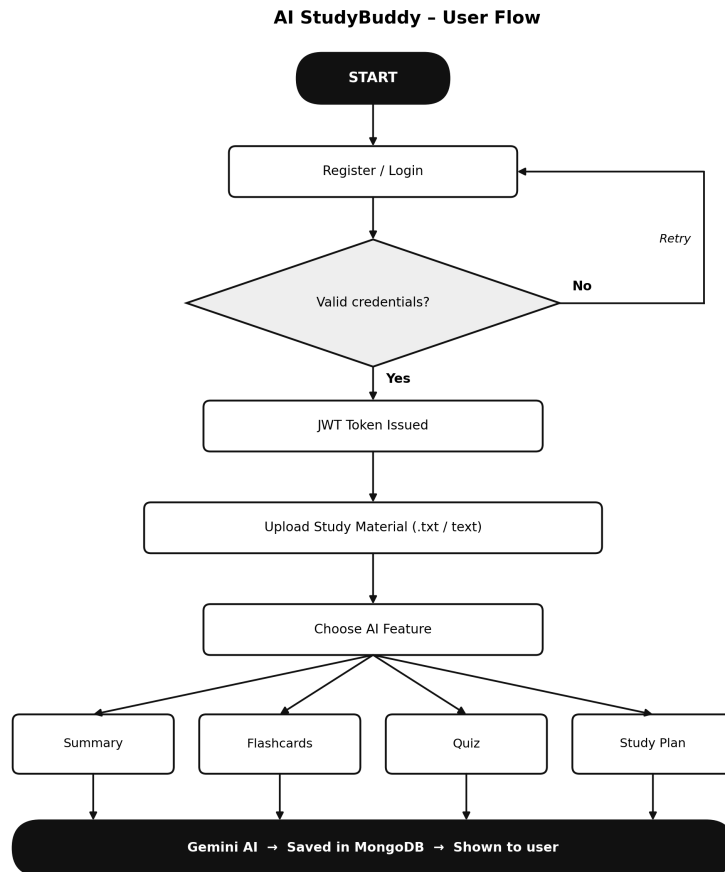


Figure 3: User Flow Diagram

10. Project Setup and Configuration

Step 1: Initialise the project

```
npm init -y
```

Step 2: Install dependencies

```
1. npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
2. npm install --save-dev nodemon
3. npm install mongoose
```

Step 3: Configure the .env file

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
```

11. Backend Development

The backend is organised into separate folders for routes, controllers, middleware, models and utilities.

```
AI-STUDYBUDDY/
├── src/
│   ├── controllers/  adminController.js, authController.js,
│                       materialController.js
│   ├── middleware/   auth.js, upload.js
│   ├── models/       Material.js, User.js
│   ├── routes/       admin.js, auth.js, materials.js
│   └── utils/         db.js, gemini.js, tokens.js
├── .env
├── index.js
└── package.json
```

- **index.js:** loads .env, sets up middleware, connects MongoDB, registers routes and starts the server.
- **utils/gemini.js:** sends prompts to Google Gemini 2.5 Flash and returns the AI response.
- **middleware/auth.js:** verifies the JWT token and restricts admin routes (RBAC).
- **middleware/upload.js:** handles uploaded study files using Multer.
- **Security:** passwords are hashed with bcryptjs, access and refresh tokens are created in utils/tokens.js, and secrets are kept in .env.

12. Database Configuration and API Endpoints

MongoDB is the database and Mongoose defines the schemas. The connection is created in `utils/db.js` when the server starts. The database has two collections: Users (account details and role) and Materials (uploaded content with its AI-generated results).

API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	User (Student) registration – role: student
POST	/api/auth/register	Admin registration – role: admin
POST	/api/auth/login	User (Student) login – returns JWT tokens
POST	/api/auth/login	Admin login – returns JWT tokens
POST	/api/materials/upload	Upload study material (token required)
POST	/api/materials/:id/summarize	Generate AI summary
POST	/api/materials/:id/flashcards	Generate AI flashcards
POST	/api/materials/:id/quiz	Generate AI quiz
POST	/api/materials/:id/study-plan	Generate AI study plan
–	/api/admin/*	Admin-only routes to manage users

13. Project Execution and API Testing

Make sure MongoDB is running and the .env file is configured. Then run:

```
npm install
npm start
```

The server connects to MongoDB and starts listening on port 5000. The APIs were tested using Postman.

1. User (Student) Registration – POST /api/auth/register

A student is registered by sending the role as student. The password is stored in encrypted form and JWT tokens are returned (201 Created).

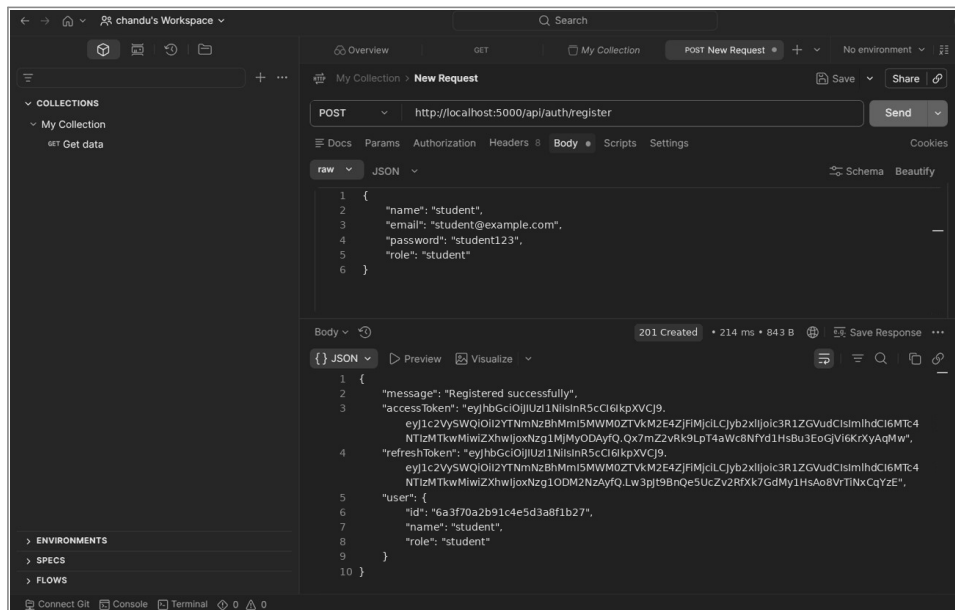


Figure 4: Student Registration API

2. Admin Registration – POST /api/auth/register

Sends name, email, password and role as admin. The password is stored in encrypted form and JWT tokens are returned (201 Created).

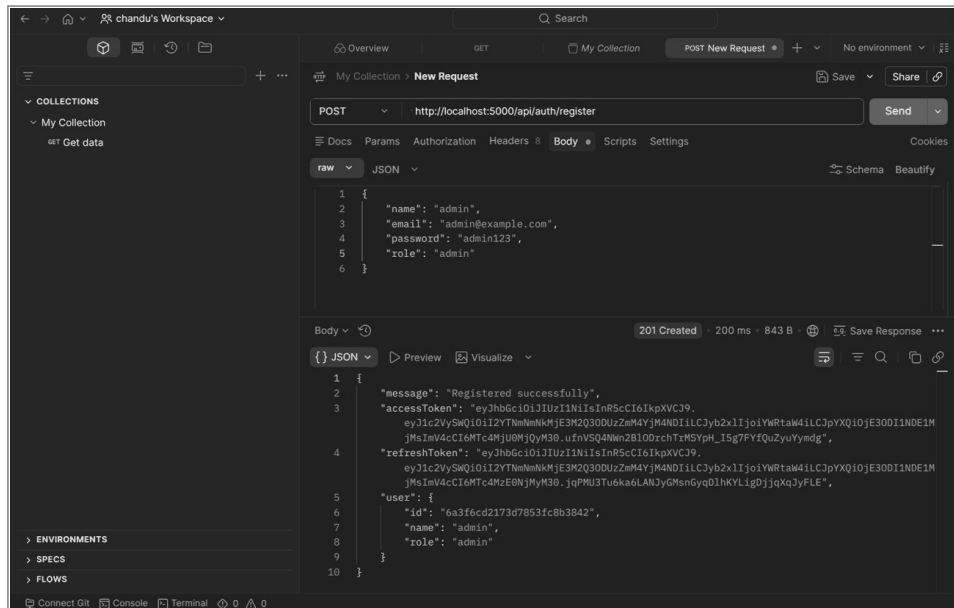


Figure 5: Admin Registration API

3. User (Student) Login – POST /api/auth/login

A student logs in with the registered email and password and receives access and refresh tokens (200 OK).

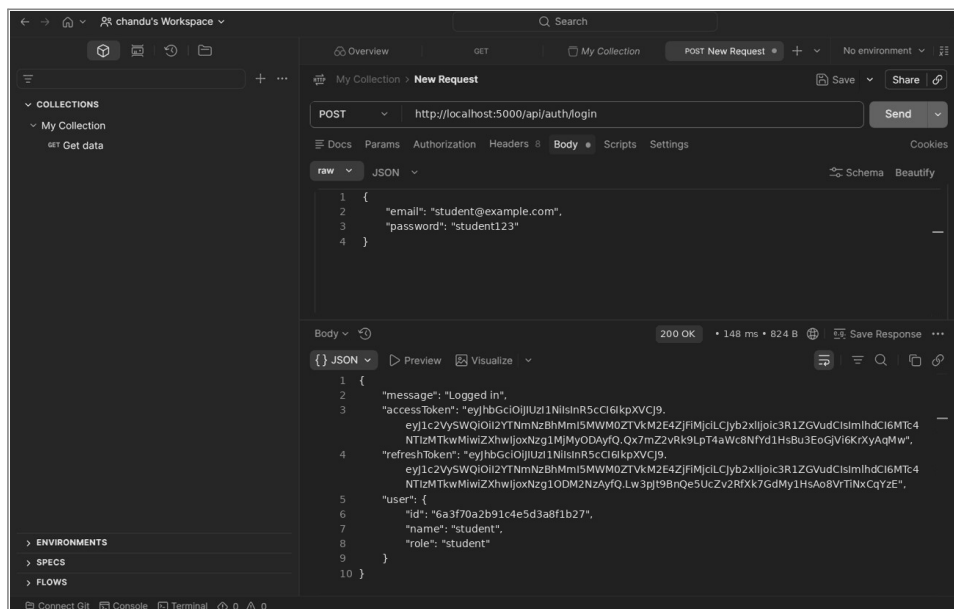


Figure 6: Student Login API

4. Admin Login – POST /api/auth/login

Verifies the email and password and returns the access and refresh tokens (200 OK).

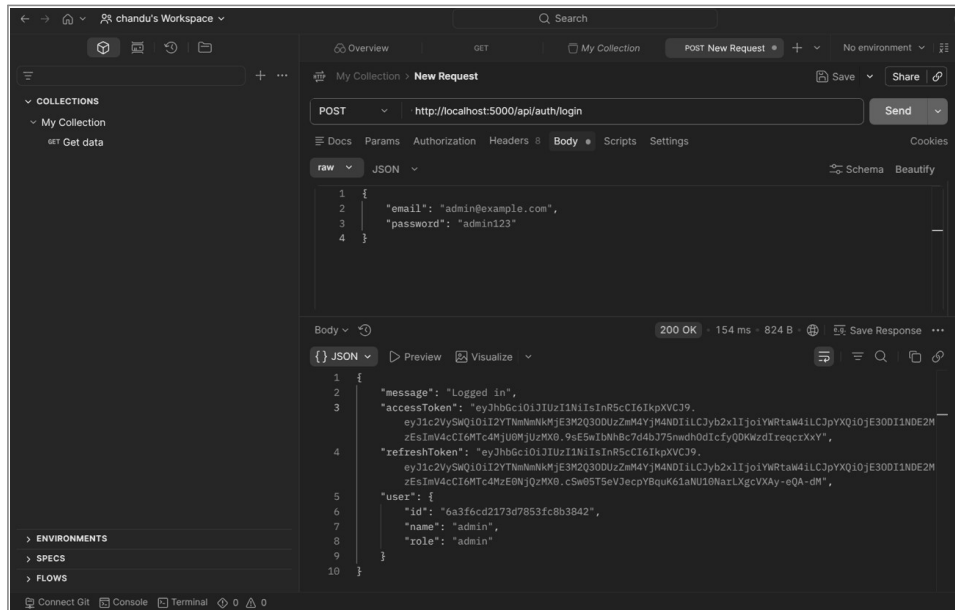


Figure 7: Admin Login API

5. Upload Study Material – POST /api/materials/upload

Uploads a text file with a title using form-data. A valid token is required (201 Created).

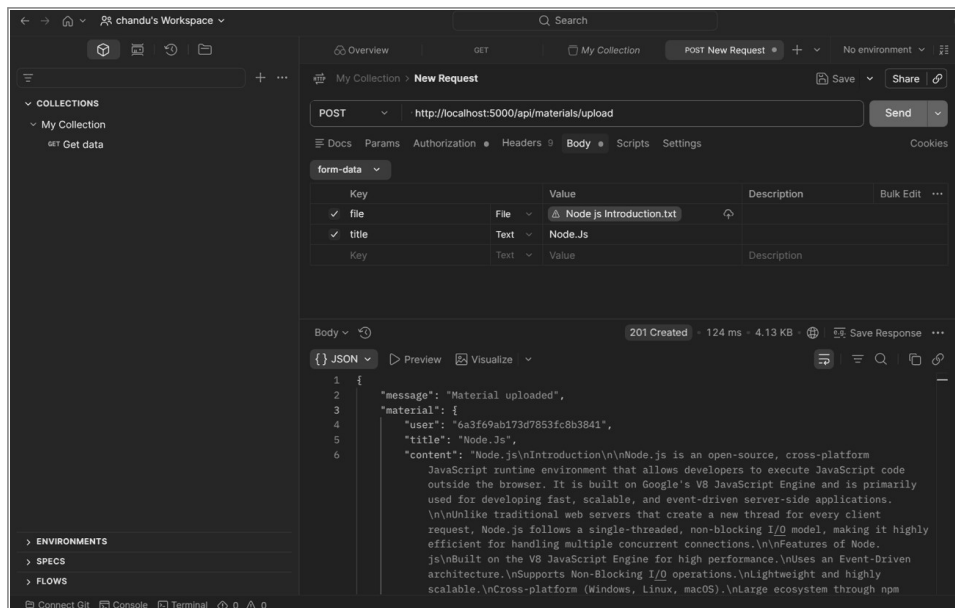


Figure 8: Upload Material API

6. Generate Flashcards – POST /api/materials/:id/flashcards

Gemini creates question–answer flashcards from the material (200 OK).

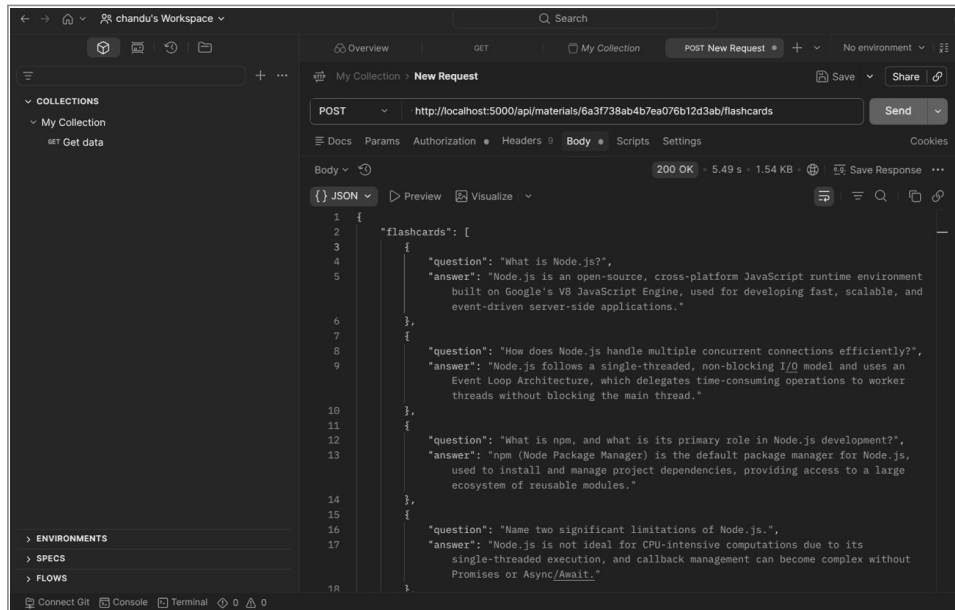


Figure 9: Flashcards API

7. Generate Quiz – POST /api/materials/:id/quiz

Creates multiple-choice questions with options and the correct answer (200 OK).

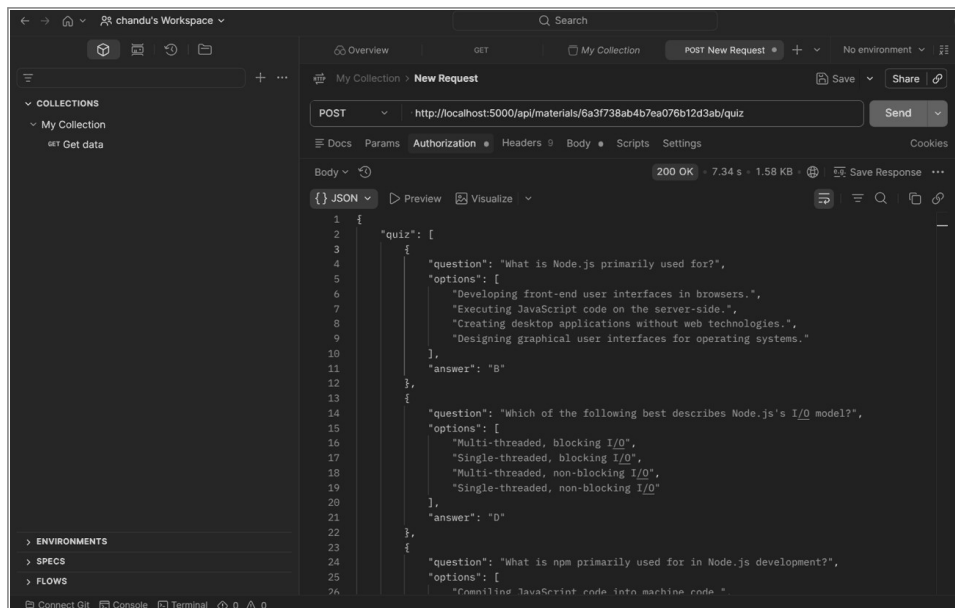


Figure 10: Quiz API

8. Generate Study Plan – POST /api/materials/:id/study-plan

Takes goal, hoursPerDay and days, and returns a personalised day-wise plan (200 OK).

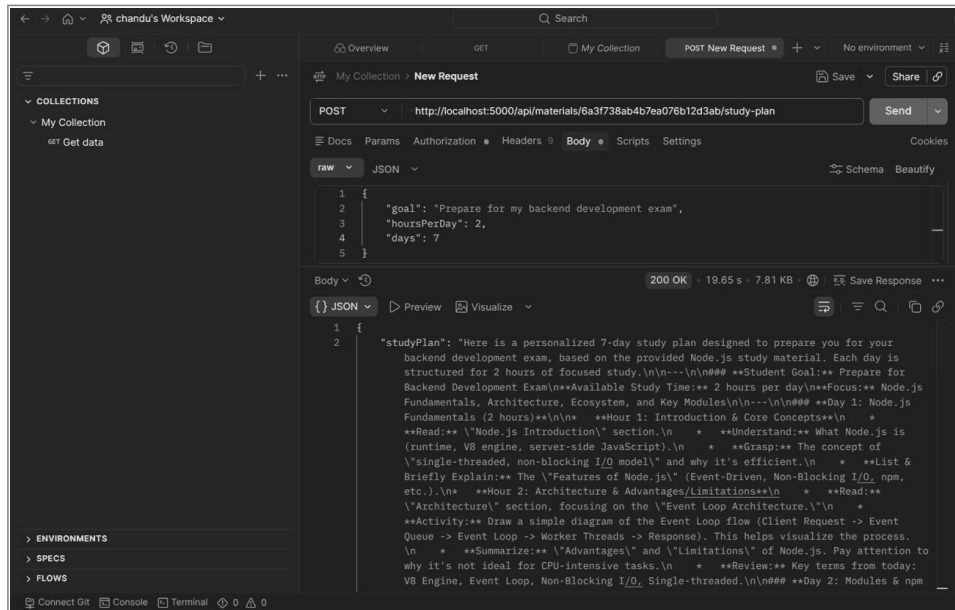


Figure 11: Study Plan API

14. Future Enhancements

- **Frontend dashboard:** a complete React interface for uploading materials and viewing AI results.
- **More file formats:** support PDF, DOCX and image (OCR) uploads along with text files.
- **Progress tracking:** quiz scores, study streaks and performance analytics for each student.
- **Smart reminders:** email or mobile notifications based on the generated study plan.
- **Multi-language support:** generate summaries and quizzes in regional languages.
- **Export options:** download summaries, flashcards and study plans as PDF.
- **Advanced admin analytics:** usage reports and AI request monitoring.
- **Cloud deployment:** host the application on the cloud for wider access.

15. Conclusion

AI StudyBuddy is a secure and useful learning platform that combines Node.js, Express.js, MongoDB, JWT authentication and the Google Gemini API. It automatically produces summaries, flashcards, quizzes and study plans, reducing the manual effort of exam preparation. The MVC-based modular design makes the system easy to maintain and extend with new AI features in the future.